HDOC DAY – 47

ALGORITHM:

Algorithm: ceil_index()

1. Start.
2. Read the size n of the array.
3. Declare a variable-length array arr[n].
4. Read the sorted array elements in ascending order.
5. Read target.
6. Pass the address of arr[], n, and target to ceil_index().
7. Inside ceil_index():
    - Set low = 0, high = n - 1, and result = -1.
    - Repeat while low <= high:
        - Calculate mid = (low + high) / 2.
        - If arr[mid] >= target:
            - Store mid in result.
            - Set high = mid - 1 to continue searching toward the left.
        - Otherwise, set low = mid + 1.
8. Return result.
9. If result == -1, display -1.
10. Otherwise, display the ceil element and its index.
11. Stop.

C PROGRAM:

```
#include <stdio.h>


// Function to find the index of ceil of target
int ceil_index(int *arr, int n, int target)
{
```

```
    int low = 0, high = n - 1;
    int result = -1;

    while (low <= high)
    {
        int mid = (low + high) / 2;

        // arr[mid] can be a possible ceil
        if (*(arr + mid) >= target)
        {
            result = mid;

            // Search towards left for a smaller element
            // >= target or first occurrence
            high = mid - 1;
        }
        else
        {
            // Ceil must be on the right side
            low = mid + 1;
        }
    }

    return result;
}

int main()
{
```

```
int n, target;

printf("Enter the size of array: ");
scanf("%d", &n);

int arr[n];

printf("Enter %d elements in ascending order:\n", n);

for (int i = 0; i < n; i++)
{
   scanf("%d", &arr[i]);
}

printf("Enter target: ");
scanf("%d", &target);

// Passing address of array to the function
int index = ceil_index(arr, n, target);

if (index == -1)
{
   printf("-1\n");
}
else
{
   printf("Ceil of %d: %d at index %d\n",
        target, arr[index], index);
```

```
    }


    return 0;
}
```

TEST CASE TABLE:

Test	Array	Target	Expected Ceil	Expected Index	Output
1	5 7 7 8 8 10	6	7	1	Ceil of 6: 7 at index 1
2	5 7 7 8 9 10	8	8	3	Ceil of 8: 8 at index 3
3	5 7 7 8 8 10	11	None	-1	-1
4	2 4 6 8 10	5	6	2	Ceil of 5: 6 at index 2
5	3 3 3 5 7	3	3	0	Ceil of 3: 3 at index 0
6	5 10 15 20	1	5	0	Ceil of 1: 5 at index 0